$ ./SHADER_LIB --present

SHADER_LIB_

100 real-time GPU shaders, rendered in your browser by Rust + WebAssembly. Browse, tweak live, learn.

🏆 Gemini AI Hackathon

Rust · wgpu · WASM

WebGPU · 60fps

demo https://shader-94o.pages.dev

code https://github.com/tangivis/SHADER

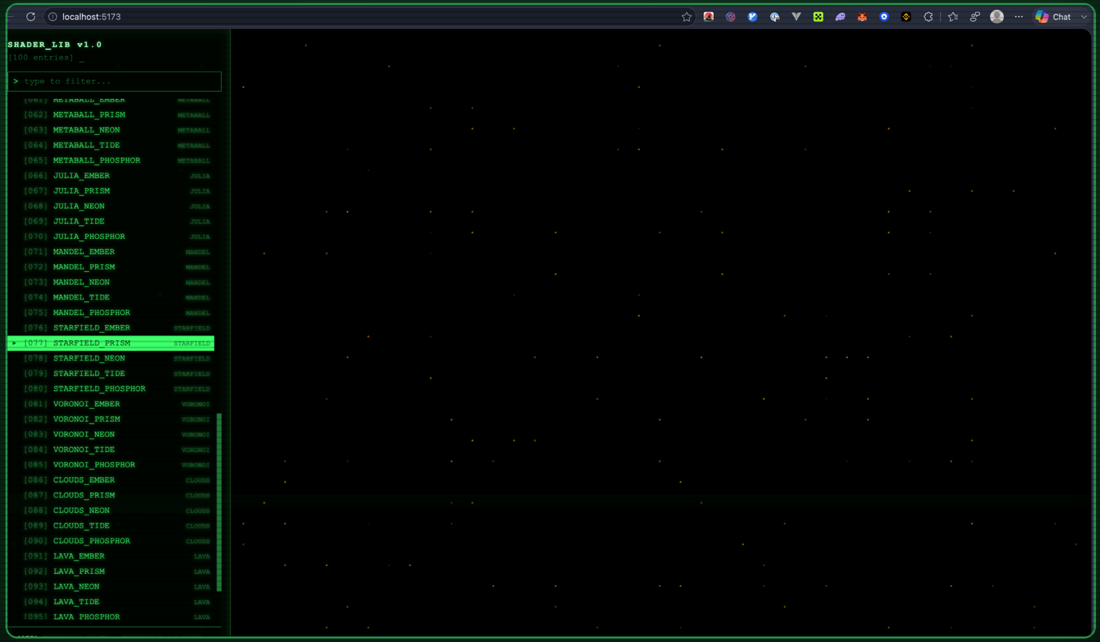
Shaders make the web's most stunning visuals —
and almost nobody can write them.

GPU fragment shaders produce mesmerizing, infinitely-varied real-time art — but authoring one means mastering WGSL / GLSL math: SDFs, noise fields, raymarching, polar folds.

SHADER_LIB turns that wall into a playground — a living gallery of 100 GPU shaders you can browse, tweak in real time, and read the source of. Zero install. Runs 100% in the browser.

A CRT-terminal shader gallery.

- ▶ 100 real-time shaders @ 60fps
- ▶ 20 visual families × 5 palettes
- ▶ Browse by keyboard, filter, random
- ▶ Mouse-interactive on every shader



Every shader is a live instrument.

Drag a knob → the GPU responds instantly. No recompiling, no reloading.

— UNIVERSAL KNOBS —

SPEED
SCALE
HUE
BRIGHT

— PER-SHADER KNOBS —

JULIA → MORPH
TUNNEL → TWIST
VORONOI → JITTER
KALEIDO → FOLD

controls ↑/↓ browse R random 0 reset · type to filter

A self-validating WGSL engine, compiled to WebAssembly.

- Rust + wgpu → WASM — a fullscreen fragment-shader runtime on WebGPU
- Compiles arbitrary WGSL live and hot-swaps the GPU pipeline at 60fps
- Validates every shader with naga → 100 / 100 compile, zero broken tiles
- On a compile error, the validator returns a precise, source-annotated message

> That compile → validate → render → repair-on-error loop is exactly what you need to turn an AI's words into live GPU art.

Describe it. Gemini writes the shader. It renders live — and fixes itself.

```
"flowing purple lava, mouse repels the glow"
  ↓  Gemini → WGSL
  ↓  engine compiles + naga-validates
  ↘ compile error? → feed it back to Gemini → auto-repair
  ↓  60fps GPU art
```

The 100-shader library is the proven foundation; natural-language → live shader is the headline interaction the engine was built for.

One clean data path.

```
text / gallery pick ──▶ set_shader(WGSL) ──▶ ┌──────────────────────────┐
                                   │ Rust + wgpu (WASM) │
live knobs ──▶ set_params / set_extra ────▶ │ naga validate → pipeline → │
                                   │ rAF 60fps uniform write │
                                   └────────────┬─────────────┘
                                        ▼ <canvas> WebGPU
```

Rust 2021 | wgpu 0.20 | naga | wasm-bindgen | Vite | Cloudflare Pages

Built different.

- WASM does what JS can't — real-time shader compile + 60fps GPU render
- Self-validating corpus — every shader provably compiles (naga 100/100)
- 100% local — no backend, no API key, works offline; deploys as static files
- Folder-driven — drop a .wgsl with a @knob header → it appears with sliders
- Demo-ready — screenshot-worthy by design, instant load

$ open https://shader-94o.pages.dev

Try it now_

demo https://shader-94o.pages.dev

code https://github.com/tangivis/SHADER

Built for the Gemini AI Hackathon by カキ · 0xtang